

DAY 2 — PYTHON FUNDAMENTALS

If-Else • Comparison Operators • Logical Operators • Conditional Logic

MODULE Python

PRACTICE 20 if-else questions

TASK Grade Calculator

What You'll Learn Today

- How to make decisions in code using if, elif, and else
- Reviewing comparison operators (==, !=, >, <, >=, <=)
- Reviewing logical operators (and, or, not) inside conditions
- Nesting conditions and combining multiple checks
- Avoiding common if-else mistakes
- Building a practical Grade Calculator application

1 IF-ELSE — Making Decisions in Code

What is If-Else?

If-else lets your program choose between different actions depending on whether a condition is True or False. Think of it as a fork in the road: the program checks a condition, and takes one path or the other.

Syntax

```
if condition:
    # code runs if condition is True
else:
    # code runs if condition is False
```

Example 1 — Basic If-Else

```
# Basic if-else
age = 20

if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")

# Output: You are an adult
```

Example 2 — If, Elif, Else (Multiple Conditions)

```
# Checking multiple conditions with elif
marks = 75

if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 60:
    print("Grade: C")
else:
    print("Grade: F")

# Output: Grade: B
```

Key Points About If-Else

- The colon : after the condition is required
- Code inside if/elif/else must be indented (4 spaces)
- elif is short for "else if" and lets you check more conditions
- Python checks conditions top to bottom and stops at the first True one
- else is optional — you can use if alone

2 COMPARISON OPERATORS — A Quick Refresh

Comparison operators are used inside if conditions to compare two values. They always return True or False.

Operator	Meaning	Example	Result
==	Equal to	10 == 10	True
!=	Not equal to	10 != 5	True
>	Greater than	10 > 5	True
<	Less than	10 < 5	False
>=	Greater or equal	10 >= 10	True
<=	Less or equal	10 <= 5	False

Example — Comparison Inside If-Else

```
temperature = 35

if temperature > 30:
    print("It's hot outside")
else:
    print("Weather is pleasant")

# Output: It's hot outside
```

3 LOGICAL OPERATORS — Combining Conditions

Logical operators let you combine multiple conditions inside a single if statement.

Operator	Meaning	Example	Result
and	Both must be True	True and False	False
or	At least one True	True or False	True
not	Opposite	not True	False

Example — Logical Operators in If-Else

```
age = 22
has_id = True

# and - both conditions must be True
if age >= 18 and has_id:
    print("Entry allowed")
else:
    print("Entry denied")

# Output: Entry allowed
```

Example — Nested If-Else

```
marks = 82
attendance = 78

if marks >= 40:
```

```

if attendance >= 75:
    print("Result: Pass")
else:
    print("Result: Detained (low attendance)")
else:
    print("Result: Fail")

# Output: Result: Pass

```

Tip — Combine instead of nesting

The nested example above can also be written in one line using and:

if marks >= 40 and attendance >= 75: — both styles work, choose whichever is more readable.

4 PRACTICAL EXAMPLE — Grade Calculator

Task: Build a calculator that takes marks as input and prints the corresponding grade using if-elif-else.

Grade Calculator — Step by Step

```

# Grade Calculator - Step by Step

# Step 1: Get input from user
marks = float(input("Enter your marks (out of 100): "))

# Step 2: Decide the grade using if-elif-else
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 40:
    grade = "D"
else:
    grade = "F"

# Step 3: Display result
print("\n==== GRADE REPORT =====")
print(f"Marks: {marks}")
print(f"Grade: {grade}")
if grade == "F":
    print("Status: Fail")
else:
    print("Status: Pass")
print("=====")

```

Sample Output

```

Enter your marks (out of 100): 82

==== GRADE REPORT =====
Marks: 82.0
Grade: B
Status: Pass
=====

```

Breaking Down the Grade Calculator

- `float(input())`: gets marks from the user as a decimal number
- if-elif-else chain: checks marks against grade thresholds, top to bottom
- Nested check: a second if-else decides Pass/Fail status based on the grade
- `print()` with f-strings: displays a clean, formatted report

5 COMMON MISTAKES TO AVOID

1. Forgetting the Colon

✗ **Wrong** `if age > 18`

✓ **Correct** `if age > 18:`

2. Using = Instead of ==

✗ **Wrong** `if age = 20:`

✓ **Correct** `if age == 20:`

3. Incorrect Indentation

✗ **Wrong** code not indented under `if/else`

✓ **Correct** always indent with 4 spaces under `if/elif/else`

4. Using elif After the Final Else

✗ **Wrong** placing `elif` after an `else` block

✓ **Correct** order must always be `if → elif → else`

5. Comparing Strings and Numbers

✗ **Wrong** `if input("Age: ") > 18:`

✓ **Correct** `if int(input("Age: ")) > 18:`

6 QUICK REFERENCE GUIDE

Concept	Syntax	Example
Basic If	<code>if condition:</code>	<code>if age > 18:</code>
If-Else	<code>if / else</code>	<code>if x > 5: ... else: ...</code>
Multiple Conditions	<code>if / elif / else</code>	<code>if / elif / elif / else</code>
Combine with AND	<code>cond1 and cond2</code>	<code>age >= 18 and has_id</code>
Combine with OR	<code>cond1 or cond2</code>	<code>age < 18 or is_vip</code>
Negate with NOT	<code>not condition</code>	<code>not is_locked</code>
Nested If	<code>if inside if</code>	<code>if a: if b: ...</code>

Key Takeaways

- If-else lets your program make decisions and choose between paths
- `elif` checks additional conditions when the first `if` is `False`
- Comparison operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) build the conditions
- Logical operators (`and`, `or`, `not`) let you combine multiple conditions
- Conditions can be nested, but combining with `and/or` is often cleaner
- Always double-check indentation and colons — Python is strict about both

Next Steps

- Practice: try the 20 if-else practice questions for Day 2
- Build: create a complete Grade Calculator (A/B/C/D/F with Pass/Fail)
- Experiment: write small programs combining comparison and logical operators
- Upload: save your Grade Calculator code on GitHub